Feature Requirements Document (FRD) v1.0

Portfolio Web Application Enhancements

Product: Personal Portfolio Website
Purpose: Enhance the existing portfolio application with interactive features that improve usability, personalization, and dynamic content delivery while demonstrating applied JavaScript capabilities.

1. Project Filter Bar

Feature Name

Project Filtering by Category

Objective

Provide users with the ability to filter displayed projects based on technology stack or category in order to improve navigation and content discoverability.

Functional Description

A filter control panel will be introduced above the project grid. Each control represents a category such as MERN, AI, or Full Stack. When a filter option is selected, only projects matching the selected category should remain visible within the grid.

The filtering logic should operate on the client side and update the UI dynamically without reloading the page.

User Interaction

1. User navigates to the **Projects** section.
2. A filter bar is displayed above the project cards.
3. User selects a category filter.
4. The displayed project cards update to reflect the selected category.

Technical Concepts to be utilized

- JavaScript arrays and objects

- Conditional logic
- DOM manipulation
- Event listeners

Data Requirements

A structured JavaScript dataset representing project metadata.

Attributes:

- Project name
- Category
- Technology stack
- Links to demo and repository

Acceptance Criteria

- Filter options must be visible above the project grid.
- Selecting a category dynamically updates visible projects.
- Selecting "All" restores the full project list.
- Filtering must occur without page reload.

2. Project Search Capability

Feature Name

Project Search

Objective

Enable users to search portfolio projects by keyword or technology to quickly locate relevant work samples.

Functional Description

A search input field will be added above the project section. As users type, the system filters visible project cards in real time by matching project titles or technologies associated with each project.

User Interaction

1. User locates the search input above the project list.
2. User enters a keyword.
3. The project list updates dynamically to show matching entries.

Technical Concepts to be utilized

- String comparison
- Array filtering
- DOM updates
- Input event handling

Data Requirements

Project metadata must include searchable fields:

- Project title

- Technologies used

Acceptance Criteria

- Search updates project results dynamically while typing.
- Search is case-insensitive.
- Clearing the search input restores all projects.

3. Dynamic Skills Rendering

Feature Name

Skills Section Dynamic Rendering

Objective

Improve maintainability and scalability of the skills section by generating skill cards dynamically from a JavaScript data model.

Functional Description

Skill information will be stored in a structured JavaScript dataset. When the page loads, the application dynamically generates skill cards based on the data provided.

This approach eliminates redundant HTML and simplifies future updates.

User Interaction

Users simply view the Skills section as part of the portfolio. Rendering occurs automatically during page initialization.

Technical Concepts to be utilized

- JavaScript objects
- Arrays
- Iteration
- Dynamic DOM element creation

Data Requirements

Each skill object should include:

- Skill name
- Description

- Icon or identifier

Attributes:

- name
- description
- category

Acceptance Criteria

- Skill cards must be rendered dynamically at runtime.
- New skills can be added by updating the JavaScript dataset only.
- No duplicated static HTML should be required.

4. Contact Form Validation

Feature Name

Client-side Contact Form Validation

Objective

Ensure user-submitted contact information is valid before processing the request.

Functional Description

The contact form modal must validate user inputs before allowing submission. Validation will verify that required fields are present and correctly formatted.

Validation errors should be displayed inline to guide the user.

User Interaction

1. User opens the contact modal.
2. User enters name and email.
3. User submits the form.
4. Validation feedback appears if inputs are invalid.

Technical Concepts to be utilized

- Form element access
- Event handling
- Conditional validation logic
- Preventing default form submission

Data Requirements

Form inputs include:

- Name
- Email
- Optional message

Acceptance Criteria

- Name field must not be empty.
- Email must match valid email format.
- Validation feedback must be displayed without page reload.
- Valid input results in a success confirmation message.

5. Theme Preference Persistence

Feature Name

User Theme Preference (Light/Dark Mode)

Objective

Allow users to toggle between visual themes and persist their selection across sessions.

Functional Description

A theme toggle control will be introduced in the navigation area. When activated, the application updates the page theme and stores the user preference locally so that the selected theme remains active on subsequent visits.

User Interaction

1. User clicks the theme toggle.
2. The UI switches between themes.
3. The preference is retained after page refresh.

Technical Concepts to be utilized

- Browser local storage
- DOM class manipulation
- Conditional logic

Data Requirements

A stored key-value pair representing the selected theme.

Example:
theme = dark or light

Acceptance Criteria

- Theme toggle must update the UI instantly.
- Theme selection must persist across page reloads.
- Default theme must load if no preference exists.

6. Developer Statistics Section

Feature Name

Developer Snapshot (Dynamic Data)

Objective

Provide dynamic developer metrics such as project count and coding activity through asynchronous data retrieval.

Functional Description

A new section will display developer-related statistics retrieved from an external or simulated API endpoint. Data will be fetched asynchronously and rendered into the UI.

The feature demonstrates dynamic content loading.

User Interaction

1. User scrolls to the Developer Snapshot section.
2. Data loads automatically or on user interaction.

Technical Concepts to be utilized

- Fetch API
- JSON parsing
- Asynchronous operations
- Error handling

Data Requirements

API response structure should contain:

- number of projects

- number of commits
- coding activity statistics

Acceptance Criteria

- Data must load dynamically via Fetch.
- Loading state should be visible during retrieval.
- Errors must be handled gracefully.

7. Language Localization

Feature Name

Portfolio Language Switcher

Objective

Enable users to switch the interface language of the portfolio between English, Hindi, and Kannada.

Functional Description

A language selection control will allow users to switch between supported languages. Interface labels will update dynamically based on the selected language.

The system should maintain a structured translation dataset and update DOM text elements accordingly.

User Interaction

1. User selects a language from a dropdown.
2. Interface labels update immediately.

Technical Concepts to be utilized

- Localization data structures
- DOM text updates
- Event handling
- Optional browser language detection

Data Requirements

A translation dataset mapping interface labels to supported languages.

Example elements translated:

- Projects
- Skills
- Contact
- Hero tagline

Acceptance Criteria

- Language dropdown must display three options.
- Switching language updates interface labels instantly.
- Default language loads on page initialization.

Summary

These features collectively enhance the portfolio application by introducing:

- interactive project discovery
- improved data-driven UI architecture
- form validation
- user preference persistence
- dynamic API data integration
- multilingual interface support

Modular Folder Structure

```
portfolio-project/
|
├── index.html
|
├── assets/
|   ├── images/
|   |   └── img.jpg
|   |
|   └── icons/
|
├── css/
|   └── custom.css
|
├── js/
|   ├── main.js
|   |
|   ├── data/
|   |   ├── projects.js
|   |   ├── skills.js
|   |   ├── translations.js
|   |   └── stats-config.js
|   |
|   ├── features/
|   |   ├── modal.js
|   |   ├── skills-render.js
|   |   ├── project-filter.js
|   |   ├── project-search.js
|   |   ├── contact-validation.js
|   |   ├── theme-toggle.js
|   |   ├── developer-stats.js
|   |   └── language-switcher.js
|
└── mock/
    └── developer-stats.json
```